# Gout MLOps — Pronóstico de demanda

TP Final · Operaciones de Aprendizaje Automático II · CEIA–FIUBA

Oct 9, 2026 · @Andrea Ferenaz

## Resumen

Se puso en producción el modelo de pronóstico de demanda de Gout (XGBoost, RMSE 2,70 en test 2025) con un ciclo MLOps completo que corre en Docker Compose: Airflow orquesta ETL, entrenamiento y monitoreo; MLflow versiona el modelo con promoción champion/challenger; y la predicción se sirve por REST, GraphQL, gRPC y streaming con Kafka.

El trabajo integra las capas S1 (REST), S2 (GraphQL), S3 (gRPC), S4 (streaming), S5 (data lake en MinIO) y S7 (monitoreo de drift y degradación). El monitoreo cierra el ciclo: si detecta drift o degradación, dispara el reentrenamiento, y el champion/challenger impide reemplazar el modelo en producción por uno peor.

Todo se validó levantado con Docker en GitHub Codespaces: los tres protocolos devuelven el mismo pronóstico, el streaming procesó 2.160 pedidos de junio 2025 con 189 alertas (8,8 %), y el MAE online (1,83) coincide con el MAE de test (1,92).

## Caso de negocio y datos

Gout es una cadena de pastelería sin TACC con 9 sucursales. Cada día, cada sucursal pide a planta las unidades de cada producto; un pronóstico confiable reduce faltantes y desperdicio de productos perecederos. El modelo parte de la memoria de especialización, donde XGBoost fue el modelo ganador frente a Random Forest y MLP.

Los datos son pedidos reales 2023–2025: 27 archivos por sucursal y año, 426.321 registros, 101 series (96 nombres de producto, con ECLAIR y PAN repetidos como variantes). Antes de publicarlos se anonimizaron:

- Sucursales renombradas S01–S09 con una permutación aleatoria, que no revela el ranking de ventas.
- Muestra estratificada de 20 SKUs (P001–P020) entre los 78 con venta en al menos el 50 % de los días, repartida en terciles de volumen alto, medio y bajo.
- Nombres de producto eliminados; los mapeos quedan fuera del repositorio.

El dataset del TP tiene 84.420 registros: 56.340 de entrenamiento (2023–2024) y 28.080 de test (2025), con partición cronológica estricta.

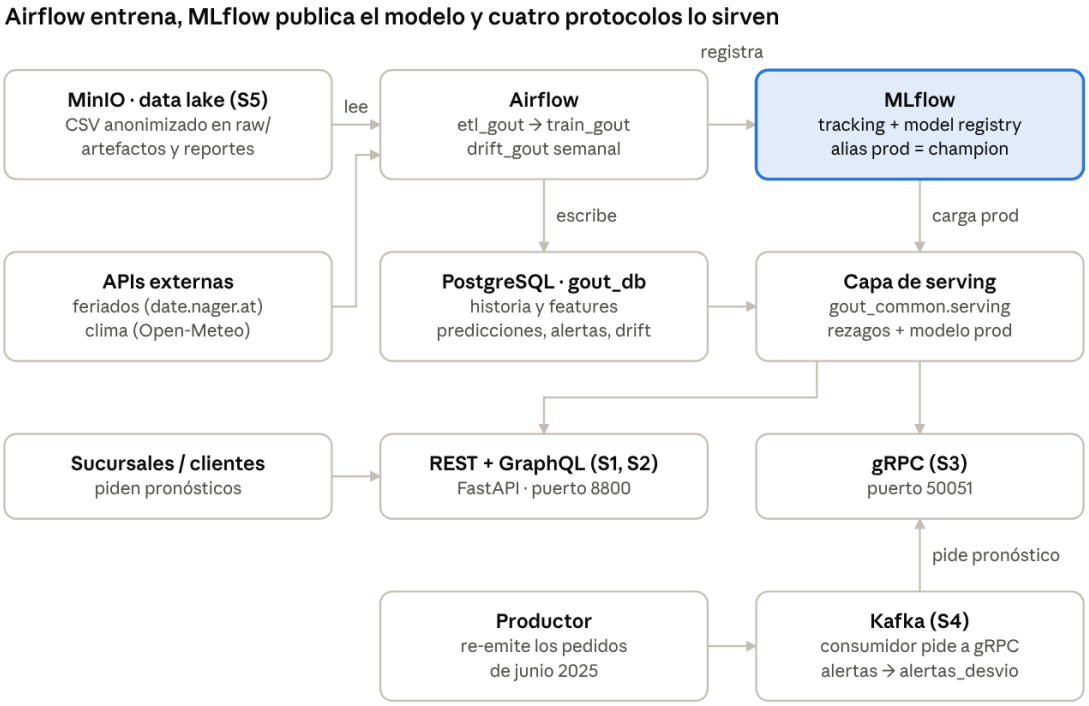
| Atributo | Origen |
|---|---|
| SKU, SUCURSAL | Codificados ordinalmente |
| DIA_SEMANA, MES, AÑO | Calendario |
| STOCK, DEVOLUCIONES | Informados por la sucursal al hacer el pedido |

| Atributo | Origen |
|---|---|
| LAG_1, LAG_2, ROLLING_MEAN_3 | Rezagos del pedido por serie |
| ES_FERIADO | API date.nager.at (feriados nacionales) |
| TEMP_MAX, TEMP_MIN | API Open-Meteo, Buenos Aires |

El objetivo es TOTAL, las unidades despachadas, que coincide con el pedido en el 100 % de los registros.

## Arquitectura

Doce servicios en un solo `docker-compose.yaml`, organizados en perfiles (`all`, `airflow`, `api`, `grpc`, `streaming`); la base es la de MLOps1 (Airflow, MLflow, MinIO y PostgreSQL) y sobre ella se montan las capas de serving y streaming.




arquitectura · servicios y flujo de datos

Airflow lee los datos de MinIO y las APIs externas, escribe historia y features en PostgreSQL y registra cada modelo en MLflow. La capa de serving carga el modelo con alias `prod` y la historia de la base, y la exponen REST, GraphQL y gRPC; el consumidor de Kafka usa gRPC para evaluar cada pedido.

## Pipeline: ETL, entrenamiento y champion/challenger

El modelo en producción obtiene RMSE 2,70, MAE 1,92 y $R^2$ 0,77 sobre 2025, y solo se reemplaza si un modelo nuevo mejora ese RMSE.

**ETL (`etl_gout`).** Tres tareas: `extraer` lee el CSV desde MinIO, valida columnas, duplicados y cantidades negativas, y lo guarda en `gout_db.ventas_historicas`; `externas` descarga feriados y clima; `features` arma los atributos y la partición temporal. Con el parámetro `disparar_entrenamiento` encadena `train_gout`.

**Features compartidas.** La ingeniería de atributos vive en un solo módulo (`gout_common.features`) que usan tanto el entrenamiento como el serving. Así se evita el *training-serving skew*: se verificó que 200 filas de 2025 armadas por el camino de predicción dan exactamente las mismas features que el de entrenamiento.

**Modelo.** Un `Pipeline` de scikit-learn con la codificación ordinal, la imputación de rezagos con la media de train y el `StandardScaler` adentro, seguido del XGBoost con los hiperparámetros de la memoria (500 árboles, profundidad 6, tasa 0,05). Todo el preprocesamiento viaja con el modelo en MLflow.

**Entrenamiento (`train_gout`).** Registra en MLflow parámetros, métricas, métricas por sucursal e importancia de atributos, y guarda el modelo en el registry como `gout_demand_model`. Luego evalúa el modelo nuevo y el vigente sobre el mismo test: el nuevo recibe el alias `prod` solo si tiene menor RMSE, y cada versión queda marcada como *champion*, *rechazado* o *retirado*. Si hubo promoción, avisa a la API para que recargue el modelo.

| Sucursal | Registros (TOTAL > 0) | RMSE | MAE | $R^2$ |
|---|---|---|---|---|
| S07 | 2.887 | 3,95 | 2,83 | 0,70 |
| S04 | 2.911 | 3,40 | 2,43 | 0,71 |
| S01 | 2.854 | 3,28 | 2,34 | 0,69 |
| S02 | 2.889 | 2,87 | 2,07 | 0,70 |
| S03 | 2.865 | 2,41 | 1,76 | 0,72 |
| S05 | 2.834 | 2,25 | 1,64 | 0,67 |
| S09 | 2.824 | 2,10 | 1,52 | 0,67 |
| S08 | 2.856 | 1,89 | 1,37 | 0,68 |
| S06 | 2.793 | 1,72 | 1,25 | 0,63 |

STOCK concentra el 83 % de la importancia del modelo, seguido de ROLLING_MEAN_3 (7 %). La calidad del pronóstico depende, entonces, de que la sucursal informe bien su stock al pedir.

Las métricas no son comparables con las de la memoria (RMSE 2,36), que usó las 101 series y no la muestra de 20.

## Serving: REST, GraphQL y gRPC

Los tres protocolos devuelven exactamente el mismo pronóstico porque comparten una única capa de lógica (`gout_common.serving`); la diferencia de latencia entre ellos es de pocos milisegundos.

Esa capa carga el modelo con alias `prod`, calcula los rezagos con las últimas tres fechas de la serie en `gout_db`, toma feriados y clima de la base, y registra cada predicción en `gout_db.predicciones` con el protocolo y el tiempo del modelo. Cada 60 segundos verifica si el alias `prod` apunta a otra versión y la recarga, así gRPC también se actualiza cuando `train_gout` promueve un modelo.

| Protocolo | Interfaz | Uso pensado |
|---|---|---|
| REST (FastAPI) | `POST /predict`, `POST /predict/batch`, `POST /model/reload` | Integración general; batch = pedido diario de una sucursal |
| GraphQL (Strawberry) | query `pronostico(sucursal, fecha, items)` | Un cliente pide exactamente los SKUs y campos que necesita |
| gRPC | `Predict`, `PredictBatch`, `Reload` | Comunicación interna; lo usa el consumidor de Kafka |

Las entradas son fecha, sucursal, SKU, stock y devoluciones. Sucursales o SKUs inexistentes devuelven 404 en REST, `NOT_FOUND` en gRPC y un error de GraphQL.

Latencia medida sobre 200 pedidos de 2025, desde el mismo cliente:

| Protocolo | Media (ms) | p50 (ms) | p95 (ms) |
|---|---|---|---|
| gRPC | 84,9 | 72,9 | 118,7 |
| REST | 86,0 | 75,3 | 123,8 |
| GraphQL | 89,6 | 79,7 | 128,6 |

De esos tiempos, unos 63 a 69 ms son el modelo y la consulta a la base, comunes a los tres (`MS_MODELO` en el registro de predicciones). El costo propio del transporte es de 10 a 20 ms: gRPC es el más liviano por usar Protocol Buffers sobre HTTP/2, y GraphQL el más pesado por parsear la consulta.

## Streaming con Kafka: alertas de pedido atípico

Sobre 2.160 pedidos de junio 2025, el sistema emitió 189 alertas (8,8 %), el 88 % de ellas por sobrepedido, y el MAE online (1,83) quedó en línea con el de test (1,92).

**Escenario.** Cada mensaje es el pedido que una sucursal carga para una fecha, con su stock y devoluciones. El **productor** re-emite, fecha a fecha, los pedidos reales del período elegido al tópico `ventas_gout`, con clave `sucursal|sku` para conservar el orden de cada serie. El **consumidor** pide el pronóstico de cada pedido por gRPC y calcula el desvío entre lo pedido y lo esperado.

**Regla de alerta.** Si el desvío supera el mayor entre 4 unidades y el 50 % del pronóstico, publica una alerta de **SOBREPEDIDO** o **SUBPEDIDO** en el tópico `alertas_desvio`, para revisar el pedido antes del despacho. Todas las comparaciones, con o sin alerta, quedan en `gout_db.evaluacion_online`, que es el insumo del monitoreo. El consumidor confirma la lectura en Kafka recién después de guardar en la base, así una caída no pierde pedidos.

| Sucursal | Sobrepedido | Subpedido |
|---|---|---|
| S02 | 26 | 5 |
| S04 | 26 | 4 |
| S07 | 23 | 5 |
| S01 | 17 | 9 |
| S03 | 14 | 1 |
| S09 | 14 | 2 |
| S06 | 13 | 0 |
| S05 | 12 | 6 |
| S08 | 11 | 1 |

Dos lecturas para el negocio:

- **El sesgo es hacia el sobrepedido.** XGBoost con error cuadrático tiende a la media y se queda corto en los picos. Parte de esas alertas son picos de demanda reales que el modelo no anticipa, no errores de la sucursal.
- **S07 concentra los desvíos más grandes:** 7 de las 10 alertas mayores (por ejemplo, 51 unidades pedidas contra 29 pronosticadas). Coincide con que es la sucursal de peor RMSE en test.

## Monitoreo de drift y degradación

El DAG semanal `drift_gout` no detectó drift ni degradación sobre los datos reales y no reentrenó; ante un cambio simulado en el patrón de pedidos, sí lo detectó y disparó el reentrenamiento.

**Drift de datos.** Para cada atributo se calcula el PSI (*Population Stability Index*) entre la ventana reciente (los últimos 30 días con pedidos evaluados) y una **referencia estacional**: las filas de entrenamiento de los mismos meses. Comparar junio contra todo el año daría drift en temperatura por pura estacionalidad. Feriados y calendario no se monitorean, porque los fija el calendario y no los datos.

$$PSI = \sum_i (q_i - p_i) \ln \frac{q_i}{p_i}$$

Con $p_i$ y $q_i$ la proporción de la referencia y de la ventana actual en cada intervalo (cuantiles de la referencia; por categoría en variables discretas). Umbrales: menos de 0,1 estable, de 0,1 a 0,2 moderado, 0,2 o más drift.

**Degradación.** Compara el MAE online (pedido real contra pronóstico, desde `evaluacion_online`) con el MAE de test del modelo en `prod`, con 20 % de tolerancia.

**Decisión.** Si hay drift en al menos el 30 % de los atributos o el modelo degrada, dispara `etl_gout` y luego `train_gout`. Cada corrida genera un reporte HTML en MinIO y una fila en `gout_db.metricas_drift`.

| Escenario | Atributos con drift | MAE online / test | Decisión |
|---|---|---|---|
| Ventana real (junio 2025) | Ninguno en pedidos ni stock | 1,91 / 1,92 | Sin acción |
| Simulado: stock +80 %, pedidos +40 % | 7 de 8 | — | Reentrenar |

En el escenario simulado el reentrenamiento corrió, pero como los datos de entrenamiento no cambiaron, el modelo nuevo empató con el vigente y quedó rechazado: el champion/challenger protegió lo que estaba en producción.

## Validación en Docker

El stack completo se levantó con `docker compose --profile all up` en GitHub Codespaces (4 núcleos, 16 GB) y cada componente se probó de punta a punta.

| Componente | Prueba | Resultado |
|---|---|---|
| Servicios | `docker compose ps` | Postgres, MinIO, MLflow, Airflow, API, gRPC y Kafka en `Up`, los que tienen chequeo en `healthy` |
| `etl_gout` | Corrida manual con `disparar_entrenamiento` | 5 tareas OK; encadenó `train_gout` |
| `train_gout` | 3 corridas | v1 en `prod`; v2 y v3 empataron y no se promovieron (`recargar_api` salteada) |
| MLflow | Experimento `gout_demand` | 3 corridas con métricas y artefactos; versiones v1–v3 en el registry |
| REST | `POST /predict` | HTTP 200, 2 unidades, modelo v1 |
| GraphQL | Query `pronostico` con 2 SKUs | 10 y 9 unidades en una sola consulta |
| Kafka | Perfil `streaming` | Alertas de sobrepedido y subpedido en tiempo real |
| `drift_gout` | Corrida programada y manual | Sin drift; `reentrenar` salteada |

Antes de Docker, el mismo código se validó en Colab reemplazando Postgres por SQLite, MinIO por Drive y Airflow por llamadas directas, con Kafka 3.8 real.

## Decisiones, problemas y trabajo futuro

**Decisiones de diseño.**

- **Lógica fuera de Airflow.** Los DAGs solo orquestan; ETL, entrenamiento y monitoreo viven en `gout_common` y se pueden probar sin levantar Airflow.
- **MinIO local en vez de nube.** Evita credenciales y facturación. MLflow sirve los artefactos (`--serve-artifacts`), así solo ese servicio necesita las claves de MinIO.
- **PSI propio en vez de Evidently.** La versión compatible con el stack no instala en Python 3.13 y las nuevas cambiaron la API; el cálculo propio tiene unas 40 líneas y es transparente.
- **REST y GraphQL en el mismo servicio,** gRPC aparte: comparten modelo y lógica, y la comparación de protocolos es justa.

**Problemas encontrados al llevarlo a Docker.**

| Problema | Causa | Solución |
|---|---|---|
| `pull access denied for minio/minio` | MinIO retiró sus imágenes de Docker Hub (sept. 2026) | Imagen `cgr.dev/chainguard/minio` y buckets creados con un script en Python |
| MLflow *unhealthy* | MLflow 2.22 no funciona con SQLAlchemy 2.1 | Fijar `sqlalchemy>=2.0,<2.1` |
| `to_sql` falla en Airflow | pandas 2.3 no funciona con el SQLAlchemy 1.4 de Airflow 2.10 | Fijar pandas 2.1.4 en la imagen de Airflow |
| Airflow redirige a `localhost` | Detrás del proxy de Codespaces | `AIRFLOW__WEBSERVER__ENABLE_PROXY_FIX` |
| MLflow 2.17 no instala en Colab | Sin pyarrow compatible con Python 3.13 | Actualizar a MLflow 2.22.1 |

**Limitaciones.**

- Muestra de 20 SKUs: las métricas no son comparables con las de la memoria.
- `STOCK` pesa el 83 % del modelo; si la sucursal no lo informa bien, el pronóstico se degrada.
- El streaming reproduce datos históricos; en operación real, los pedidos llegarían desde el sistema de la sucursal.
- En la memoria, agrupar por nombre de producto mezclaba las variantes de ECLAIR (4) y PAN (3) en los rezagos; en el TP cada SKU es una serie propia.

**Trabajo futuro.**

- **Aprendizaje federado (S6):** cada sucursal como cliente que entrena con sus propios pedidos sin compartirlos.
- Una función de pérdida que penalice más quedarse corto (cuantílica), para reducir el sesgo hacia el sobrepedido.
- Reentrenar con ventanas móviles e incorporar 2025 al entrenamiento.

## Cómo ejecutarlo

Con Docker local, o sin instalar nada desde GitHub Codespaces (el repo trae `.devcontainer/` con una máquina de 4 núcleos y 16 GB).

1. Copiar `.env.example` a `.env` (en Codespaces se crea solo) y dejar `data/pedidos_anon.csv` en el repo.
2. Levantar el stack: `docker compose --profile all up -d --build` y esperar a que `docker compose ps` muestre todo `healthy`.
3. En Airflow (puerto 8080, `airflow`/`airflow`), activar `etl_gout` y `train_gout`, y correr `etl_gout` con `disparar_entrenamiento = true`.
4. Verificar en MLflow (puerto 5005) el modelo `gout_demand_model` con alias `prod`.
5. Probar la API en el puerto 8800: `/docs` para REST y `/graphql` para GraphQL; gRPC escucha en 50051.
6. Levantar el streaming con `docker compose --profile streaming up -d` y ver las alertas con `docker compose logs consumer`.
7. Activar `drift_gout` y correrlo; el reporte queda en MinIO (consola en el puerto 9001).